
CSE Web Data Collection

How the project works, in detail

Written for 24ada076 - Nuhan

DA 2009 - Data Collection Methods II

Assignment 2 (Group Project)

University of Colombo

Covers the whole project - the architecture, the pipeline, and every file - as well as the three files you wrote.

Your 50 questions are in the shared team guide.

Contents

How to use this guide	4
The project in one page	5
The four numbers to remember	5
The story in five sentences	5
Why the project is built this way	6
What went wrong with the ordinary approach	6
Static and dynamic content	6
Proving the library was not the problem	6
The way in	6
How the whole thing fits together	7
The files and what depends on what	7
An honest gap, in case you are asked	7
The pipeline, from button to file	8
Where things end up on disk	8
Your file 1 - src/ethics.py	9
ScraperLog - the record	9
Scraper - reading the rules	9
Scraper - the waiting	9
Scraper - fetch(), the one door	9
Your file 2 - src/static_scraper.py	11
What it measures	11
The methods, and what each returns	11
The sitemap, and why it is in the file	11
Your file 3 - src/pdf_extractor.py	12
Why two libraries	12
Deciding whether a page is a scan	12
The None trap	12
What the file details give you	12
What everybody else's files do	13
25ada072 - Pasindu: api_client.py and selenium_scraper.py	13
25ada073 - Sasini: crawler.py, cleaner.py and the Scrapy spider	13

25ada141 - Salaama: ocr_extractor.py, storage.py and analysis.py	13
How the screen is wired up	14
Shared state	14
Buttons	14
Running it, and the demonstration	15
Starting the app	15
Running each part on its own	15
Before the viva	15
A three minute run through the app	15

How to use this guide

This guide covers the whole project, not only your own files. You are presenting it and you own the ethics section, so you are the most likely person to be asked about how the parts fit together.

It is arranged so you can read it in three passes:

1. **Sections 2 to 4** - what the project does, why it is built this way, and how a request travels from the button to the screen. Read this first. It is what you would say if somebody asked you to explain the project in two minutes.
2. **Sections 5 to 7** - your three files, walked through in order, with the real code. This is the detail you need for questions about your own part.
3. **Sections 8 to 10** - what everybody else's files do, how the screen is wired up, and how to run each piece. Read this so no question about the project can catch you out.

Your 50 questions are in the shared team guide, in the section headed 24ada076 - Nuhan. This document is the explanation behind those answers.

The project in one page

We collect share prices, company details and announcements from the Colombo Stock Exchange, clean the result, and save it as CSV, Excel and JSON files.

The four numbers to remember

Figure	Value	Where it comes from
Text requests finds on the CSE page	about 24 characters, no tables	<code>python -m src.static_scraper</code>
Companies in one request	around 290	<code>/api/tradeSummary</code>
Delay between our requests	1.5 seconds	<code>config.DELAY</code>
Section blocked by robots.txt	<code>/cgi-bin/</code> only	<code>https://www.cse.lk/robots.txt</code>

The story in five sentences

1. We tried `requests` and `BeautifulSoup`, and got almost nothing, because the CSE page is built by JavaScript.
2. We opened the site with F12 and watched the Network tab, which showed the page fetching its own data from `/api/` addresses.
3. Those addresses return JSON, so we read them directly - one request gives every company.
4. Announcements link to PDF documents, which we download and read with `pdfplumber` and `PyPDF2`.
5. One of those documents is a scan with no text inside, so that one goes through OCR.

Why the project is built this way

What went wrong with the ordinary approach

The course teaches: download the page with requests, parse it with BeautifulSoup. On CSE that produces this:

```
Status      : 200          <- the request worked perfectly
HTML size   : 25,071 bytes <- there is plenty of HTML
Words found: 24 characters
Tables      : 0
```

Nothing failed. The request succeeded, BeautifulSoup parsed the HTML without complaint. There is simply nothing in the file to find.

Static and dynamic content

	Static page	Dynamic page
Where the content is	Already inside the HTML the server sends	Added by JavaScript after the page loads
Does requests see it?	Yes	No - it never runs JavaScript
How to check	View Page Source shows the content	View Page Source is empty, but Inspect shows the content
CSE	-	This one

The practical test takes ten seconds. **View Page Source** shows what the server sent; **Inspect** shows the page as it is now. If the numbers are in Inspect but not in View Source, it is dynamic.

Proving the library was not the problem

The obvious follow-up question is whether we just used BeautifulSoup badly. So `src/static_scraper.py` also points the same library at CSE's sitemap, an ordinary file with no JavaScript:

```
15 pages read from the sitemap without any trouble

Page                                Updated
https://www.cse.lk/                 daily
https://www.cse.lk/about-us/        monthly
https://www.cse.lk/market-summary/   daily
```

Same library, same methods, perfect result. That isolates the problem to the page.

The way in

Week 1 gave the fix as well as the problem: use the Network tab. Opening the site with F12 showed the page requesting its own data:

```
POST https://www.cse.lk/api/tradeSummary    -> 200, JSON
GET  https://www.cse.lk/api/allSecurityCode -> 200, JSON
```

Those replies are JSON, so pandas turns them into a table in one line. No login is needed, and `robots.txt` does not ask us to stay away from them.

The ethical argument, which is the one that matters. Loading the page makes CSE send the HTML, the JavaScript, the fonts and the images, and then the browser calls that same data address at the end of it anyway. Reading the address directly skips all of that work. It is less load on their server, not more.

How the whole thing fits together

The files and what depends on what

```
app.py                                the screen, 7 tabs
|
+-- src/ethics.py                     Scraper.fetch() <-- ALL requests
|                                     pass through here
|   +--> robots.txt check
|   +--> 1.5 second wait
|   +--> requests.Session
|   +--> logs/request_log.csv
|   +--> data/cache/*.json
|
+-- src/api_client.py                 uses Scraper -> the market data
+-- src/crawler.py                    uses Scraper -> downloads PDFs
+-- src/static_scraper.py             uses Scraper -> requests + BS4
|
+-- src/selenium_scraper.py           drives its own browser
+-- src/pdf_extractor.py              reads files already on disk
+-- src/ocr_extractor.py              reads files already on disk
+-- src/cleaner.py                    pure pandas, no internet
+-- src/analysis.py                   pure plotly, no internet
+-- src/storage.py                    writes files out
```

The important shape here: the four files at the top touch the internet, and every one of them goes through Scraper. Everything below works on data that is already on the computer.

An honest gap, in case you are asked

`src/selenium_scraper.py` does **not** go through Scraper. It drives a browser, and the browser makes its own requests, so our `fetch()` function never sees them.

If somebody points that out, the honest answer is: that is true. What we do instead is give the browser the same User-Agent, load one page rather than crawling, and check by hand that the page is allowed. It is a demonstration script that runs a handful of times, not part of the collection pipeline. But it is a real exception to the rule, and it is better to say so than to claim otherwise.

The pipeline, from button to file

This is the sequence to have ready if you are asked to explain how the project works. Every tab follows it.

```
1. You click a button in app.py
   |
2. app.py calls something like  api.trade_summary()
   |                           (src/api_client.py)
   |
3. that calls  self.call("tradeSummary")
   |           which looks up GET or POST in config.ENDPOINTS
   |
4. that calls  Scraper.fetch(url, method="POST", ...)
   |           (src/ethics.py)  <-- THE ONE DOOR
   |
   |           a. can_fetch(url)    is robots.txt happy?
   |           b. wait()            sleep until 1.5s has passed
   |           c. session.post()    send it, with our User-Agent
   |           d. log_request()     write a row to the CSV log
   |           e. save_copy()       keep a copy of the reply
   |
5. the JSON reply comes back as a Python list of dictionaries
   |
6. get_records() finds the list, make_dataframe() builds the table
   |
7. the table goes back to app.py and onto the screen
   |
8. optionally: clean() -> check() -> chart -> export()
```

Where things end up on disk

Folder	What goes there	Written by
data/cache/	A copy of every reply, named after the address	src/ethics.py
data/pdfs/	The announcement documents we downloaded	src/crawler.py
data/images/	Page images made for OCR, plus each cleaning step	src/ocr_extractor.py
data/exports/	The CSV, Excel and JSON files	src/storage.py
logs/request_log.csv	One row per request, ever	src/ethics.py

Your file 1 - src/ethics.py

This is the most important file in the project, and it is the one the assignment weights most heavily. It has two classes.

ScraperLog - the record

Two jobs. It keeps the messages shown on screen, and it writes every request to logs/request_log.csv. That CSV is the evidence for how the data was collected - if anyone asks whether we hammered their server, that file is the answer.

```
def log_request(self, method, url, status, allowed):
    self.request_count += 1
    new_file = not config.REQUEST_LOG.exists()

    with open(config.REQUEST_LOG, "a", newline="",
              encoding="utf-8") as f:
        writer = csv.writer(f)
        if new_file:
            writer.writerow(["time", "method", "url", "status",
                            "robots_allowed", "delay_seconds"])
        writer.writerow(...)]
```

Note the "a" mode - it appends, so the log builds up across runs rather than being overwritten each time.

Scraper - reading the rules

read_robots() downloads the file and hands it to RobotFileParser, which is part of Python's standard library and already understands the format. We feed it the text we downloaded rather than letting it fetch the address itself, so the rules come from exactly the bytes we can show on screen.

```
def read_robots(self):
    try:
        response = requests.get(config.ROBOTS_URL, ...)
        self.robots_text = response.text

        self.robots = RobotFileParser()
        self.robots.parse(self.robots_text.splitlines())
    except Exception as e:
        self.robots = None          # fail closed, not open
```

If it cannot be read we set self.robots = None, and can_fetch() then returns False for everything. If we cannot see the rules we do not guess in our own favour.

Scraper - the waiting

wait() measures rather than blindly sleeping. If parsing the last reply already took a second, we only need another half second - the server got its 1.5 seconds either way.

```
def wait(self):
    since_last = time.time() - self.last_request_time
    if since_last < self.delay:
        time.sleep(self.delay - since_last)
    self.last_request_time = time.time()
```

Scraper - fetch(), the one door

Everything comes through here. The order of the four steps is the thing to remember, because the permission check being first is what makes the ethics claim true rather than aspirational.

```
def fetch(self, url, method="GET", data=None,
          save_as=None, want="json"):

    # 1. are we allowed?
    allowed, reason = self.can_fetch(url)
    if not allowed:
        self.log.log_request(method, url, "skipped", allowed)
        return {"ok": False, "error": ...}

    # 2. wait
    self.wait()

    try:
        # 3. send it
        if method == "POST":
            response = self.session.post(url, data=data, ...)
        else:
            response = self.session.get(url, ...)

        # 4. record it
        self.log.log_request(method, url,
                             response.status_code, allowed)

        if response.status_code != 200:
            return {"ok": False, "error": ...}

        result = response.json()
        if save_as:
            self.save_copy(save_as, result)
        return {"ok": True, "data": result, ...}

    except Exception as e:
        # fall back to the saved copy if we have one
        if save_as:
            saved = self.load_copy(save_as)
            if saved is not None:
                return {"ok": True, "data": saved,
                        "from_file": True}
        return {"ok": False, "error": str(e)}
```

Three design decisions worth being able to defend

- **It returns a dictionary instead of raising.** This runs behind a web page. An uncaught error would take the whole app down; an `ok: False` can be shown neatly on screen.
- **The permission check is the first statement.** Not somewhere in the middle, and not optional. It is not possible to send a request that skips it, because this is the only function that sends requests.
- **The saved copy is a fallback, not a shortcut.** We always try live first. The copy is only used when the request fails - which is what keeps a demonstration working if the venue internet drops.

Your file 2 - src/static_scraper.py

This is the requests and BeautifulSoup part. It does not collect the project's data, and it is in the project deliberately - it is the reasoning behind the design, written as code you can run.

What it measures

scrape_page() downloads a page and counts what BeautifulSoup can find in it - the size of the HTML, the number of readable characters, the number of tables and links.

```
soup = BeautifulSoup(html, "html.parser")

title = soup.title.get_text(strip=True)
words = soup.get_text(strip=True)
tables = soup.find_all("table")
links = soup.find_all("a")
```

The methods, and what each returns

Method	What it does	Example
find()	The first matching tag	soup.find('title')
find_all()	Every matching tag, as a list	soup.find_all('a')
select()	Every match, using a CSS selector	soup.select('div.price')
select_one()	The first CSS match	soup.select_one('#total')
get_text()	The words inside a tag	tag.get_text(strip=True)
get()	One attribute of a tag	tag.get('href')

strip=True on get_text() removes the whitespace around the text. Without it you get newlines and indentation from the HTML source mixed in with the words.

The sitemap, and why it is in the file

scrape_sitemap() reads CSE's sitemap.xml - a file where the site lists its own pages for programs like ours. It is ordinary static content, and BeautifulSoup reads it perfectly.

It is there to answer the obvious challenge to the failure above. Same library, same methods, working result - so the problem was the page, not the tool.

```
try:
    soup = BeautifulSoup(response.text, "xml")
except Exception:
    soup = BeautifulSoup(response.text, "html.parser")
```

The xml mode needs lxml installed, so there is a fallback to the built-in parser. Nothing in the project should break because of an optional library.

Your file 3 - src/pdf_extractor.py

Why two libraries

	PyPDF2	pdfplumber
What it is for	General PDF work - text, metadata, merging	Text extraction that respects the layout
Layout	Often merges lines and loses structure	Keeps the layout using the position of the text
Tables	No table support	Can detect and extract tables
We use it for	The file details - pages, author, producer	The main text extraction, and tables

Deciding whether a page is a scan

```
def page_type(text):
    length = len(text.strip())

    if length < config.OCR_TEXT_THRESHOLD:    # 50
        return "Scanned image", f"Only {length} characters"
    return "Text based", f"{length:,} characters read"
```

The threshold is 50 rather than 0. A real page of a CSE circular returns over a thousand characters and a scan returns nothing, so almost anything would work - but a scan sometimes carries a few stray characters that are not the document text, such as a header added afterwards. Requiring a real amount of text means a handful of stray letters cannot fool us into skipping OCR.

The None trap

```
text = page.extract_text() or ""
```

`extract_text()` returns `None`, not an empty string, when a page has nothing on it. Calling `.strip()` or `len()` on `None` raises an error - and the pages most likely to return `None` are exactly the scanned ones we care about. The `or ""` makes every page safe to handle the same way.

What the file details give you

PyPDF2 can read the information stored inside the PDF, which is often more revealing than people expect:

```
Pages          1
Title          not set
Author         Nirmalee Ganegoda
Created with   Microsoft Word for Microsoft 365
```

That tells you the document was typed in Word rather than scanned, which is a second clue on top of the character count.

What everybody else's files do

Enough to answer a question about any part of the project without having to hand over to somebody else.

25ada072 - Pasindu: `api_client.py` and `selenium_scraper.py`

- `call()` looks up whether an address needs GET or POST in `config.ENDPOINTS`, then hands it to your `Scraper.fetch()`.
- `get_records()` exists because the replies are not all shaped the same - some are a plain list, some wrap the list under a name like `reqTradeSummery`, some are a list inside a list.
- `make_dataframe()` turns the list of dictionaries into a table, and converts any millisecond timestamps into real dates.
- `selenium_scraper.py` opens the page in Edge, waits for the table to appear with `WebDriverWait`, takes `driver.page_source`, and parses that with `BeautifulSoup`. It only gets about 25 companies, because the table on screen is paginated.

25ada073 - Sasini: `crawler.py`, `cleaner.py` and the Scrapy spider

- `find_documents()` builds the full PDF address from the fragment the listing gives, by putting `https://cdn.cse.lk/cmt/` in front of it, and removes duplicates.
- `download_documents()` fetches them one at a time through your `Scraper`, and checks the first four bytes are `%PDF` before saving.
- `cleaner.py` converts millisecond dates into real dates, turns numbers-stored-as-text into numbers, drops columns that are empty for every company, and strips extra spaces. Then `check()` asks whether the values are believable.
- The Scrapy spider does the crawl a second time using the framework - `start()` sends the first POSTs, `parse()` follows each PDF link, `parse_document()` records the result.

25ada141 - Salaama: `ocr_extractor.py`, `storage.py` and `analysis.py`

- OCR runs in four steps: `pdfplumber` turns the page into an image, `OpenCV` converts it to grey then to black and white (plus the lecture's optional noise step, which at 1x1 changes nothing), `pytesseract` reads the letters, `pandas` tidies the result.
- `tesseract_ready()` calls `get_tesseract_version()`, because importing `pytesseract` succeeds even when the Tesseract program itself is missing.
- `storage.py` writes CSV, Excel and JSON, and puts the source inside each one - comment lines in the CSV, a second sheet in the Excel file, a details block in the JSON.
- `analysis.py` builds three charts and works out the summary figures - how many companies rose, how many fell, total turnover.

How the screen is wired up

`app.py` holds no collection logic. It calls the modules and puts the results on screen. Understanding two things is enough.

Shared state

One `State` object holds the `Scraper`, the `CSEApi`, and whatever data has been collected so far, so a table fetched on one tab can be cleaned on another. Both are created the first time they are needed, so the window opens instantly instead of waiting for `robots.txt`.

```
@property
def scraper(self):
    if self._scraper is None:
        self._scraper = Scraper()
    return self._scraper
```

Buttons

Every button follows the same pattern: a function does the work and returns values, and `.click()` says which boxes on screen those values go into, in order.

```
market_button.click(get_market_data,
                    outputs=[market_figures_html, trades_table,
                             sectors_table, addresses_table,
                             market_log])
```

So `get_market_data()` returns five things, and they fill those five boxes. If you ever add an output you must add it in both places, or Gradio raises an error about the count not matching.

Running it, and the demonstration

Starting the app

```
cd D:\DA2009\CSE-Scraping-Project
.venv\Scripts\activate
python app.py

then open http://127.0.0.1:7860
```

Running each part on its own

Each member can show their own work without launching the app:

```
python -m src.ethics           # 24ada076 - Nuhan
python -m src.static_scraper   # 24ada076 - Nuhan
python -m src.pdf_extractor    # 24ada076 - Nuhan
python -m src.api_client       # 25ada072 - Pasindu
python -m src.selenium_scraper # 25ada072 - Pasindu
python -m src.crawler          # 25ada073 - Sasini
python -m src.cleaner          # 25ada073 - Sasini
python -m src.ocr_extractor     # 25ada141 - Salaama
python -m src.storage          # 25ada141 - Salaama
python -m src.analysis         # 25ada141 - Salaama
```

Before the viva

1. Run `python warm_cache.py`. It collects everything once and saves a copy, so the demonstration works even if the internet does not.
2. Check the OCR tab actually runs. It needs the Tesseract program installed, not just the Python package.
3. Open the Ethics tab and press the button, so `robots.txt` is already on screen when you start talking.
4. Have `logs/request_log.csv` ready to show if anybody asks about load.

A three minute run through the app

Time	Tab	What to say
0:00	Home	What we collect, and that the site is JavaScript so the ordinary approach found nothing.
0:30	Ethics	Press the button. Show the real robots.txt, the one blocked section, and that we wait anyway.
1:15	Market Data	Press the button. Around 290 companies from one request.
1:50	Announcements	Download a few PDFs, then read the text out of one.
2:20	OCR	The scanned one. Show the three cleaning images and the recovered text.
2:45	Data & Export	Clean, chart, and save. Mention the source is inside every file.